LAPORAN PRAKTIKUM 1
OPENMP
PEMROSESAN PARALEL

Dosen Pengampu : Huda Ubaya, M.T.



Disusun Oleh :
M. Daffa Ramadhan
09011382429133

DEPARTEMEN SISTEM KOMPUTER
FAKULTAS ILMU KOMPUTER
UNIVERSITAS SRIWIJAYA
2026

BAB I
PENDAHULUAN

1.1 Latar Belakang

Komputasi yang berat dapat dipercepat dengan membagi pekerjaan ke beberapa inti prosesor yang bekerja bersamaan. Pada sistem memori bersama, OpenMP memudahkan pembagian ini melalui direktif yang ditambahkan ke program C serial. Meski begitu, percepatan tidak otomatis benar: iterasi yang dibagi harus saling bebas, variabel harus ditentukan sebagai *shared* atau *private* dengan tepat, dan pembaruan terhadap data bersama harus dijaga agar tidak terjadi *race condition*.

Praktikum ini memparalelkan dua program. Program pertama mengalikan dua matriks berukuran 1024×1024, yang bebannya merata di setiap baris hasil. Program kedua menghitung jumlah bilangan prima dari 2 sampai 10^7, yang bebannya tidak merata karena biaya memeriksa setiap bilangan berbeda-beda. Kedua program dijalankan dengan 1, 2, 4, dan 8 thread, masing-masing tiga kali, untuk mengukur waktu, speedup, efisiensi, dan pengaruh kebijakan *scheduling*.

1.2 Tujuan Praktikum

- Memparalelkan perulangan dengan `#pragma omp parallel for` dan menentukan cakupan variabel dengan benar.
- Menggabungkan hasil antar-thread dengan klausa `reduction`.
- Membandingkan `schedule(static)`, `schedule(dynamic, 1000)`, dan `schedule(guided)` secara terukur pada beban yang tidak seragam.
- Menghitung speedup dan efisiensi dari median tiga kali pengukuran, serta memeriksa kebenaran hasil terhadap versi serial dan nilai acuan 664.579.

BAB II
DASAR TEORI

2.1 OpenMP dan Direktif *parallel for*

OpenMP adalah antarmuka pemrograman paralel untuk arsitektur memori bersama, dipakai melalui direktif kompilator seperti `#pragma omp parallel for`. Direktif ini membagi iterasi sebuah perulangan ke sejumlah thread. Pembagian hanya aman jika setiap iterasi tidak bergantung pada hasil iterasi lain.

2.2 Cakupan Variabel, *Reduction*, dan *Race Condition*

Variabel yang dideklarasikan di luar region paralel bersifat *shared*, sehingga dapat dibaca dan ditulis semua thread. Variabel *private* dimiliki terpisah oleh tiap thread. Jika beberapa thread memperbarui satu variabel *shared* secara bersamaan, hasilnya bergantung pada urutan eksekusi; kondisi ini disebut *race condition*.

Klausa `reduction` menyelesaikan masalah itu untuk operasi akumulasi. Setiap thread mendapat salinan variabel sendiri, lalu salinan-salinan itu digabung di akhir region.

2.3 Kebijakan *Scheduling*

Klausa `schedule` menentukan cara iterasi dibagikan ke thread.

Tabel 1. Kebijakan *scheduling* pada OpenMP

Kebijakan	Cara kerja	Cocok untuk
static	Iterasi dibagi rata sebelum eksekusi dimulai	Biaya tiap iterasi seragam
dynamic, k	Potongan k iterasi diberikan ke thread yang sudah selesai	Biaya iterasi bervariasi
guided	Potongan awal besar, lalu mengecil selama eksekusi	Biaya bervariasi dengan overhead pembagian lebih kecil
runtime	Kebijakan dibaca dari variabel OMP_SCHEDULE	Eksperimen tanpa kompilasi ulang

2.4 Speedup dan Efisiensi

Speedup membandingkan waktu serial dengan waktu paralel, dan efisiensi membandingkan speedup dengan jumlah thread. Pada kondisi ideal, speedup sama dengan jumlah thread dan efisiensi 100%.

$$S(p) = T_{\text{serial}} / T_{\text{paralel}} \qquad E(p) = S(p) / p \times 100\%$$

dengan p jumlah thread. Setiap konfigurasi diuji tiga kali dan diwakili oleh median, supaya satu pengukuran yang menyimpang tidak ikut mempengaruhi hasil.

BAB III
METODE PELAKSANAAN

3.1 Alat, Bahan, dan Lingkungan Pengujian

Pengujian berjalan di Ubuntu pada Windows Subsystem for Linux 2 (WSL2). Program ditulis dalam bahasa C dan dikompilasi dengan GCC yang mendukung OpenMP. Bahan praktikum adalah kerangka `matmul.c` dan `prima.c` dari petunjuk praktikum serta rancangan dekomposisi perkalian matriks dari tugas A04. Data lingkungan diambil dari keluaran `lscpu`, `free -h`, dan `gcc --version`.

Tabel 2. Lingkungan pengujian

Komponen	Keterangan
Prosesor	12th Gen Intel Core i5-12500H
CPU logis (lscpu di WSL2)	16 (dilaporkan sebagai 8 core × 2 thread per core)
Memori	7,6 GiB (bagian yang dialokasikan WSL2)
Sistem operasi	Ubuntu 26.04 LTS pada WSL2
Versi GCC	15.2.0 (Ubuntu 15.2.0-16ubuntu1)
Opsi kompilasi	-O2 -fopenmp

Angka 8 core dan 16 CPU logis mengikuti pembacaan `lscpu` di dalam WSL2. Spesifikasi i5-12500H sendiri adalah prosesor hibrida (gabungan *performance core* dan *efficiency core*), jadi 8 core pada tampilan WSL2 merupakan penyederhanaan dan tidak menjamin semua thread berjalan di core yang setara. Memori 7,6 GiB adalah bagian yang dialokasikan WSL2, bukan seluruh memori laptop.

3.2 Perkalian Matriks

Kerangka `src/matmul.c` dijalankan lebih dulu apa adanya untuk memastikan keluarannya `benar=ya` dengan waktu serial dan paralel yang hampir sama. Setelah itu, fungsi `matmul_paralel` diparalelkan dengan menambahkan `#pragma omp parallel for` pada perulangan terluar berindeks *i*, sesuai rancangan A04. Fungsi `matmul_serial` tidak diubah dan menjadi acuan kebenaran, yang dibandingkan melalui fungsi `sama()` dengan toleransi 10^{-9}.

Ukuran matriks yang dipakai adalah $n = 1024$. Waktu serial pada mesin ini sekitar 2,3 detik, sehingga syarat petunjuk untuk beralih ke $n = 1536$ (bila waktu serial di bawah 2 detik) tidak berlaku. Skrip `jalan_matmul.sh` menjalankan program dengan 1, 2, 4, dan 8 thread, masing-masing tiga kali, dan menambahkan keluarannya ke `hasil/matmul.txt` (12 baris).

```
gcc -O2 -fopenmp src/matmul.c -o matmul -lm
OMP_NUM_THREADS=4 ./matmul 1024
./jalan_matmul.sh 1024
```

3.3 Penghitungan Bilangan Prima

Program `src/prima.c` memeriksa setiap bilangan dari 2 sampai 10.000.000 dengan fungsi `is_prima()`. Perulangan utamanya diparalelkan dengan `#pragma omp parallel for reduction(+:jumlah) schedule(runtime)`. Klausa *reduction* menjaga penjumlahan tetap benar, sedangkan `schedule(runtime)` membuat kebijakan *scheduling* dapat diganti melalui variabel lingkungan `OMP_SCHEDULE` tanpa kompilasi ulang. Jumlah bilangan prima yang benar, yaitu 664.579, dipakai untuk memeriksa setiap hasil.

Skrip `prima.sh` menjalankan tiga kebijakan (static, dynamic,1000, guided) dengan 1, 2, 4, dan 8 thread, tiga kali per konfigurasi. Hasilnya 3 × 4 × 3 = 36 baris di `hasil/prima.txt`.

```
gcc -O2 -fopenmp src/prima.c -o prima
OMP_SCHEDULE="dynamic,1000" OMP_NUM_THREADS=4 ./prima
./prima.sh
```

3.4 Pengolahan Data

Setiap konfigurasi diwakili oleh median dari tiga pengulangan, bukan rata-rata. Untuk perkalian matriks, speedup dihitung dari median waktu serial dan median waktu paralel pada jumlah thread yang sama. Untuk bilangan prima, speedup dihitung dari median waktu 1 thread terhadap median waktu 8 thread pada kebijakan yang sama. Efisiensi dihitung sebagai speedup dibagi jumlah thread.

BAB IV
HASIL DAN PEMBAHASAN

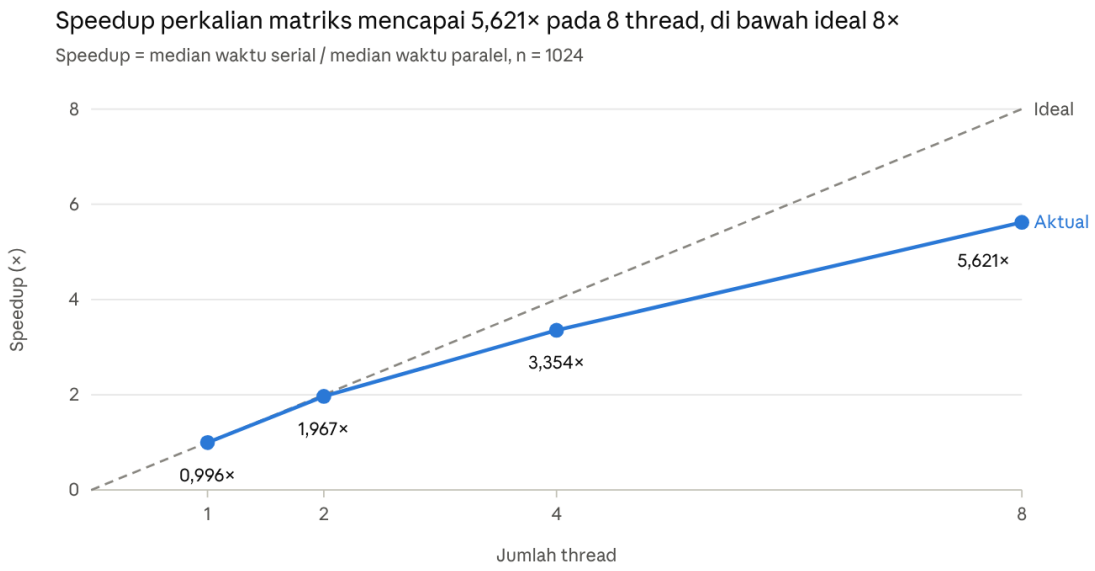
4.1 Perkalian Matriks

Semua 12 pengukuran perkalian matriks (n = 1024) menghasilkan `benar=ya`. Median waktu serial berkisar 2,34 sampai 2,43 detik. Dua pengukuran serial menyimpang (2,784 detik pada 2 thread dan 2,873 detik pada 8 thread), sehingga median lebih tepat daripada rata-rata.

Tabel 3. Perkalian matriks, n = 1024 (median tiga pengulangan, waktu dalam detik)

Thread	Waktu serial	Waktu paralel	Speedup	Efisiensi	Benar?
1	2,341	2,350	0,996	99,62%	ya
2	2,386	1,213	1,967	98,35%	ya
4	2,351	0,701	3,354	83,84%	ya
8	2,434	0,433	5,621	70,27%	ya

Speedup = waktu serial / waktu paralel; efisiensi = speedup / jumlah thread.



Gambar 1. Perbandingan speedup aktual dan ideal pada perkalian matriks (median hasil/matmul.txt, 3 pengulangan per konfigurasi, n = 1024).

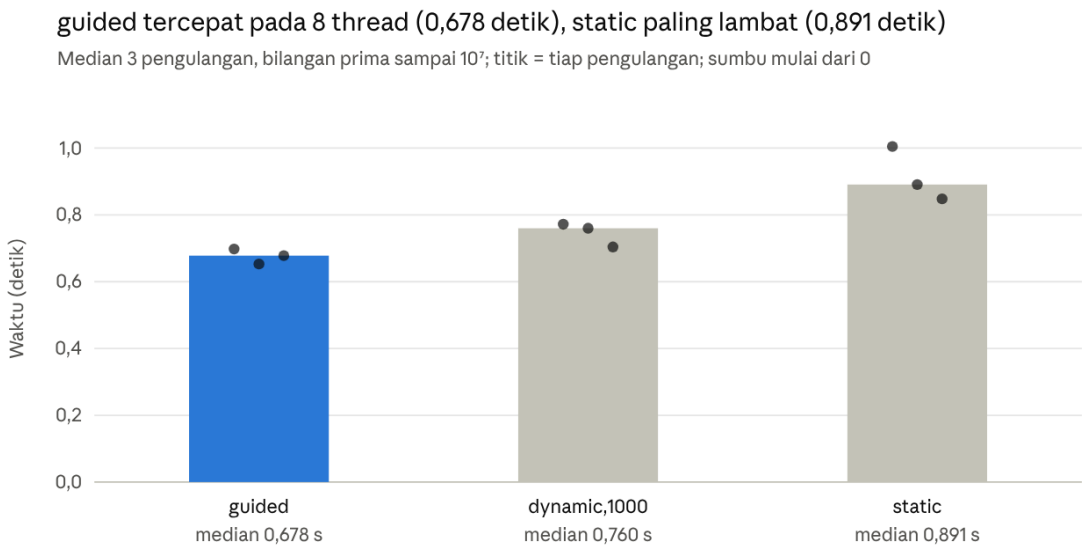
4.2 Bilangan Prima dan *Scheduling*

Semua 36 pengukuran menghasilkan 664.579 bilangan prima, sama dengan nilai acuan. Pada 1 thread ketiga kebijakan hampir sama (4,08 sampai 4,13 detik), sehingga perbedaan baru muncul ketika thread bertambah.

Tabel 4. Bilangan prima sampai 10^7, median waktu (detik) per kebijakan *scheduling*

Kebijakan	1 thread	2 thread	4 thread	8 thread	Speedup 8 thread	Efisiensi 8 thread	Hasil = 664.579?
static	4,109	2,600	1,550	0,891	4,612	57,65%	ya
dynamic, 1000	4,127	2,131	1,204	0,760	5,430	67,88%	ya
guided	4,080	2,221	1,173	0,678	6,018	75,22%	ya

Speedup 8 thread = median waktu 1 thread / median waktu 8 thread pada kebijakan yang sama.



Gambar 2. Median waktu eksekusi bilangan prima pada 8 thread untuk tiga kebijakan *scheduling* (titik = tiap pengulangan).

4.3 Pembahasan

1. Perulangan yang diparalelkan dan variabel *private*

Pada `matmul_paralel`, `#pragma omp parallel for` dipasang pada perulangan terluar berindeks *i*, sehingga tiap thread mengerjakan sekumpulan baris matriks C. Variabel yang harus *private* adalah *i*, *j*, *k*, dan *s*. Indeks *i* otomatis *private* karena menjadi variabel kendali `parallel for`, sedangkan *j*, *k*, dan *s* dideklarasikan di dalam perulangan sehingga tiap thread memiliki salinan sendiri tanpa klausa `private`. Matriks A dan B hanya dibaca, dan elemen `C[i*n + j]` yang ditulis berbeda untuk setiap *i*, jadi tidak ada dua thread yang menulis ke lokasi yang sama. Karena itu tidak terjadi *race condition*, dan seluruh baris di Tabel 3 bernilai `benar=ya`.

2. Speedup dibandingkan kondisi ideal

Speedup aktual pada 1, 2, 4, dan 8 thread adalah 0,996×, 1,967×, 3,354×, dan 5,621×, sedangkan speedup ideal adalah 1×, 2×, 4×, dan 8×. Efisiensi turun dari 99,62% (1 thread) dan 98,35% (2 thread) menjadi 83,84% (4 thread) dan 70,27% (8 thread). Dua faktor yang paling mungkin menjelaskan selisih ini:

a. Keterbatasan memori dan cache. Elemen `B[k*n + j]` diakses dengan lompatan *n* elemen untuk setiap *k*, sehingga banyak *cache miss*, dan makin banyak thread berarti makin banyak yang berebut cache bersama serta *bandwidth* memori.

b. Overhead dan kondisi mesin. Pembuatan thread, sinkronisasi di akhir region paralel, kemungkinan turunnya frekuensi CPU laptop saat banyak inti aktif, serta WSL2 dan proses lain di Windows ikut membatasi percepatan.

Praktikum ini tidak mengukur kedua faktor itu secara terpisah, jadi keduanya adalah penjelasan yang paling masuk akal, bukan hasil pengukuran.

3. Pengaruh *scheduling* pada bilangan prima

Biaya memeriksa bilangan *x* kira-kira sebanding dengan $\sqrt{x}$, sehingga bilangan besar lebih mahal daripada bilangan kecil. Pada *static*, iterasi dibagi menjadi blok berurutan yang sama besar, dan thread terakhir mendapat bilangan-bilangan terbesar. Dengan anggapan kasar bahwa biaya sebanding dengan $\sqrt{x}$, blok terakhir dari 8 blok memuat sekitar 18% total pekerjaan, padahal pembagian yang merata adalah 12,5%. Thread itu menjadi penentu waktu, sehingga speedup *static* terbatas sekitar 5,5×. Hasil terukur 4,612× (efisiensi 57,65%) berada di bawah batas kasar ini.

dynamic,1000 dan *guided* membagikan iterasi selama eksekusi berjalan sehingga beban lebih seimbang. Pada 8 thread, median waktu *static* 0,891 detik, *dynamic,1000* 0,760 detik, dan *guided* 0,678 detik, dengan speedup 4,612×, 5,430×, dan 6,018× (efisiensi 57,65%, 67,88%, dan 75,22%). Menurut data ini, *guided* terbaik pada 8 thread dan juga pada 4 thread (1,173 detik, dibandingkan 1,204 detik untuk *dynamic,1000* dan 1,550 detik untuk *static*). Pada 2 thread *dynamic,1000* sedikit lebih cepat (2,131 detik, dibandingkan 2,221 detik untuk *guided*). Alasan yang masuk akal untuk keunggulan *guided* pada thread banyak: potongan besar di awal dipakai untuk bilangan kecil yang murah, dan potongan kecil di akhir untuk bilangan besar yang mahal, dengan jumlah pembagian yang lebih sedikit daripada *dynamic,1000* (sekitar 10.000 potongan).

5. Delapan thread pada mesin ini

Petunjuk menanyakan pengaruh jika core fisik kurang dari 8. Pada mesin ini `lscpu` melaporkan 8 core dan 16 CPU logis (Tabel 2), jadi 8 thread tidak melebihi jumlah CPU logis dan tidak terjadi *oversubscription*. Meski begitu, efisiensi pada 8 thread tetap turun: 70,27% pada perkalian matriks dan 57,65% sampai 75,22% pada bilangan prima. Penurunan ini lebih sesuai dijelaskan oleh faktor pada jawaban 2 (cache, *bandwidth* memori, *overhead*) dan, pada bilangan prima, ketidakseimbangan beban. Prosesor i5-12500H juga berarsitektur hibrida dengan inti yang kecepatannya tidak sama, sehingga 8 thread belum tentu berjalan di inti yang setara, tetapi hal itu tidak diukur dalam praktikum ini. Seandainya core fisik kurang dari 8, beberapa thread harus berbagi inti dan efisiensi akan turun lebih jauh karena thread-thread itu bergantian memakai sumber daya eksekusi yang sama.

BAB V
PENUTUP

5.1 Kesimpulan

Paralelisasi perulangan terluar pada perkalian matriks 1024 × 1024 menghasilkan perhitungan yang sama dengan versi serial pada semua 12 pengukuran. Speedup naik seiring jumlah thread dan mencapai 5,621× pada 8 thread (efisiensi 70,27%), masih di bawah speedup ideal 8×.

Pada bilangan prima, seluruh 36 pengukuran dengan *reduction* menghasilkan 664.579. Pada 8 thread, *guided* memberi median waktu terendah (0,678 detik, speedup 6,018×), diikuti *dynamic,1000* (0,760 detik) dan *static* (0,891 detik). Urutan ini sesuai dengan beban per iterasi yang tidak seragam, karena *static* membebani thread terakhir dengan bilangan terbesar.

Hasil praktikum menunjukkan bahwa pembagian iterasi saja belum cukup: cakupan variabel yang benar, penggunaan *reduction*, dan pemilihan kebijakan *scheduling* yang sesuai dengan pola beban sama-sama menentukan kebenaran dan kecepatan program paralel.

5.2 Saran

Untuk beban yang tidak seragam seperti penghitungan bilangan prima, kebijakan *guided* atau *dynamic* layak dicoba lebih dulu dibanding *static*. Pengukuran berikutnya dapat diperluas dengan ukuran potongan *dynamic* yang bervariasi dan dengan pengikatan thread ke core (*thread affinity*) agar pengaruh prosesor hibrida dapat dipisahkan dari pengaruh *scheduling*.

DAFTAR PUSTAKA

Petunjuk Praktikum 1. (t.t.). *Pemrograman Shared Memory dengan OpenMP*, asesmen A05, minggu 5. Materi praktikum yang diberikan dosen.

OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface, Version 5.2*. https://www.openmp.org

LAMPIRAN
HASIL PENGUJIAN MENTAH DAN DOKUMENTASI

Lampiran A. Hasil Perkalian Matriks (`hasil/matmul.txt`)

Isi berkas apa adanya. Median pada Tabel 3 dihitung dari keluaran ini.

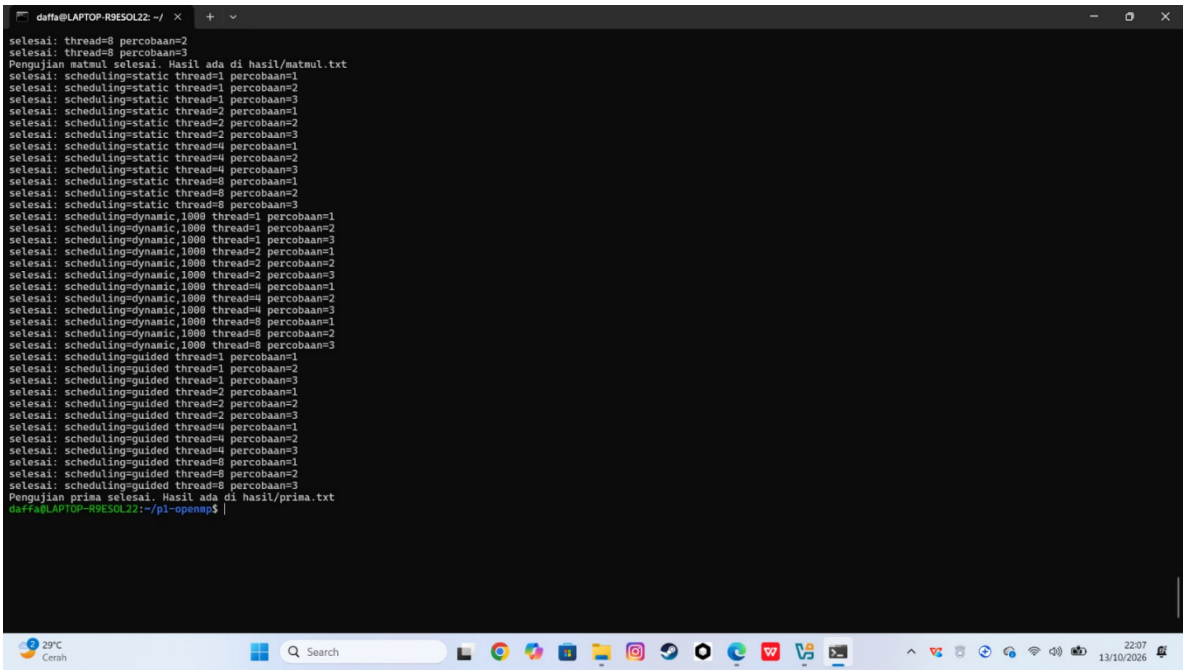
```
n=1024 thread=1 serial=2.360 paralel=2.399 benar=ya
n=1024 thread=1 serial=2.341 paralel=2.296 benar=ya
n=1024 thread=1 serial=2.312 paralel=2.350 benar=ya
n=1024 thread=2 serial=2.784 paralel=1.236 benar=ya
n=1024 thread=2 serial=2.386 paralel=1.213 benar=ya
n=1024 thread=2 serial=2.329 paralel=1.205 benar=ya
n=1024 thread=4 serial=2.348 paralel=0.751 benar=ya
n=1024 thread=4 serial=2.351 paralel=0.648 benar=ya
n=1024 thread=4 serial=2.387 paralel=0.701 benar=ya
n=1024 thread=8 serial=2.873 paralel=0.423 benar=ya
n=1024 thread=8 serial=2.434 paralel=0.433 benar=ya
n=1024 thread=8 serial=2.396 paralel=0.444 benar=ya
```

Lampiran B. Hasil Bilangan Prima (`hasil/prima.txt`)

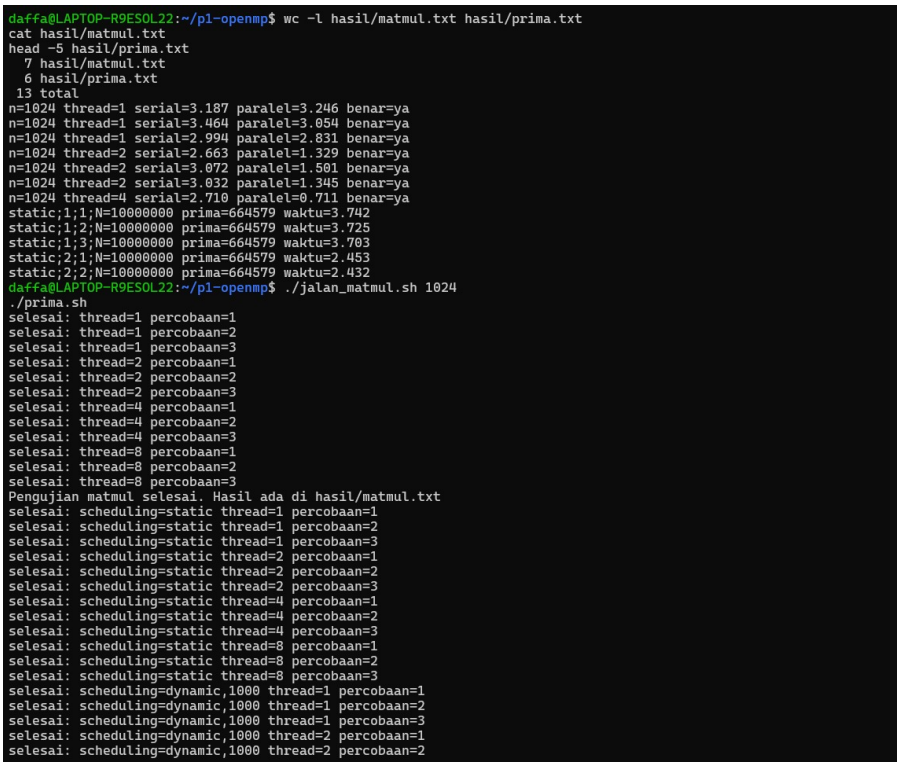
Format baris: kebijakan; jumlah thread; pengulangan; keluaran program. Median pada Tabel 4 dihitung dari keluaran ini.

```
static;1;1;N=10000000 prima=664579 waktu=4.072
static;1;2;N=10000000 prima=664579 waktu=4.109
static;1;3;N=10000000 prima=664579 waktu=4.175
static;2;1;N=10000000 prima=664579 waktu=2.607
static;2;2;N=10000000 prima=664579 waktu=2.600
static;2;3;N=10000000 prima=664579 waktu=2.582
static;4;1;N=10000000 prima=664579 waktu=1.532
static;4;2;N=10000000 prima=664579 waktu=1.550
static;4;3;N=10000000 prima=664579 waktu=1.558
static;8;1;N=10000000 prima=664579 waktu=1.005
static;8;2;N=10000000 prima=664579 waktu=0.891
static;8;3;N=10000000 prima=664579 waktu=0.848
dynamic,1000;1;1;N=10000000 prima=664579 waktu=4.179
dynamic,1000;1;2;N=10000000 prima=664579 waktu=4.127
dynamic,1000;1;3;N=10000000 prima=664579 waktu=4.110
dynamic,1000;2;1;N=10000000 prima=664579 waktu=2.131
dynamic,1000;2;2;N=10000000 prima=664579 waktu=2.137
dynamic,1000;2;3;N=10000000 prima=664579 waktu=2.124
dynamic,1000;4;1;N=10000000 prima=664579 waktu=1.160
dynamic,1000;4;2;N=10000000 prima=664579 waktu=1.204
dynamic,1000;4;3;N=10000000 prima=664579 waktu=1.241
dynamic,1000;8;1;N=10000000 prima=664579 waktu=0.772
dynamic,1000;8;2;N=10000000 prima=664579 waktu=0.760
dynamic,1000;8;3;N=10000000 prima=664579 waktu=0.704
guided;1;1;N=10000000 prima=664579 waktu=4.058
guided;1;2;N=10000000 prima=664579 waktu=4.112
guided;1;3;N=10000000 prima=664579 waktu=4.080
guided;2;1;N=10000000 prima=664579 waktu=2.200
guided;2;2;N=10000000 prima=664579 waktu=2.237
guided;2;3;N=10000000 prima=664579 waktu=2.221
guided;4;1;N=10000000 prima=664579 waktu=1.214
guided;4;2;N=10000000 prima=664579 waktu=1.173
guided;4;3;N=10000000 prima=664579 waktu=1.170
guided;8;1;N=10000000 prima=664579 waktu=0.698
guided;8;2;N=10000000 prima=664579 waktu=0.653
guided;8;3;N=10000000 prima=664579 waktu=0.678
```

Lampiran C. Dokumentasi Pelaksanaan



Lampiran Gambar 1. Pemeriksaan awal berkas hasil dan eksekusi skrip `jalan_matmul.sh` serta `prima.sh`.



Lampiran Gambar 2. Keluaran skrip pengujian bilangan prima sampai selesai.